40 Days of LeetCode Challenge

40 days, 40 problems
DAY :8

LEETCODE PROBLEM NO. 1295 : Find Numbers with Even Number of Digits

Problem Link : https://leetcode.com/problems/find-numbers-with-even-number-of-digits/description/
Easy · Digit Manipulation · Loops · Airthematic Operators
STEP BY STEP SOLUTION

Step 1. Understand the question :-

You're given an integer array nums. Your task is to count how many numbers contain an even number of digits.

Remember: we're counting the numbers that satisfy the condition, not counting all their digits.

For example, in nums = [12, 345, 2, 6, 7896]:

- 12 has 2 digits → even
- 345 has 3 digits → odd ❌
- 2 has 1 digit → odd ❌
- 6 has 1 digit → odd ❌
- 7896 has 4 digits → even

Step 2. Translating it into simpler language with examples :-

Example 1:
Input: nums = [12,345,2,6,7896]
Output: 2
Explanation:
12 contains 2 digits (even number of digits).
345 contains 3 digits (odd number of digits).
2 contains 1 digit (odd number of digits).
6 contains 1 digit (odd number of digits).
7896 contains 4 digits (even number of digits).
Therefore only 12 and 7896 contain an even number of digits.

Example 2:
Input: nums = [555,901,482,1771]
Output: 1
Explanation:
Only 1771 contains an even number of digits.

Imagine checking each number one by one and asking:

1. How many digits does this number have?
2. Is that digit count divisible by 2?
3. If yes, increase our answer by one.

That's it! We only need to count how many numbers pass the test.

Step 3. Identify the pattern

We'll use array traversal + digit counting.

Two Java operators do the important work:

Operator	Purpose	Example
%	Gets the remainder	123 % 10 = 3
/	Integer division removes the last digit	123 / 10 = 12

Repeatedly dividing a positive integer by 10 removes one digit at a time. We can count how many times we do this t o determine its number of digits.

Step 4. Build the logic

Here's the approach we'll follow.

Algorithm

1. Create a variable count to store the number of valid elements.
2. Visit every number in the array.
3. Copy the number into a temporary variable so the original value stays unchanged.
4. Count its digits by repeatedly dividing the temporary number by 10.
5. If the digit count is even, increment count.
6. Return count after checking all the numbers.

Pseudocode

```
START

  count = 0

  FOR each number in nums

    temp = number

    digits = 0

    WHILE temp > 0

      digits = digits + 1

      temp = temp / 10

    IF digits % 2 == 0

      count = count + 1

  RETURN count

END
```

Step 5. Code in Java

This is the standard arithmetic solution for LeetCode 1295.

```
class Solution {
    public int findNumbers(int[] nums) {
        int count = 0;
        for (int num : nums) {
            int temp = num;
            int digits = 0;
                    while (temp > 0) {
                                digits++;
                                temp /= 10;
                    }
                    if (digits % 2 == 0) {
                                count++;
                        }
        } return count;
    }
}
```

How to submit

1. Open LeetCode Problem 1295. Find Numbers with Even Number of Digits
2. Select Java as the programming language.
3. Enter the method in the provided Solution class, replacing the existing method if necessary.
4. Click Run to test the examples.
5. Click Submit after the tests pass.

Step 6. Explanation & dry-run

Explanation of the code :-

A. *class Solution {*

Defines the class required by LeetCode.

B. ***public int findNumbers(int[] nums) {***

The method accepts an integer array and returns an integer representing the total number of elements with an even digit count.

C. *int count = 0;*

Initializes the answer to zero. We increase it whenever we find a qualifying number.

D. *for (int num : nums) {*

The enhanced for loop visits every element in the array, one at a time.

E. *int temp = num;*

Creates a temporary copy. We will modify temp while counting digits, leaving num unchanged.

F. *int digits = 0;*

Starts the digit counter at zero.

G.
```
while (temp > 0) {
   digits++;
   temp /= 10;
}
```

- while (temp > 0) keeps running while digits remain.
- digits++ increases the digit count by one.
- temp /= 10 removes the last digit using integer division.

For example, if temp = 7896, the values become:

Iteration	temp before division	digits	temp after division
1	7896	1	789
2	789	2	78
3	78	3	7
4	7	4	0

When temp becomes 0, the loop stops. The number has four digits.

H.
```
if (digits % 2 == 0) {
   count++;
}
```

The expression digits % 2 == 0 checks whether the number of digits is even.

- If even, increment count.
- If odd, leave count unchanged.

I. *return count;*

Returns the final total after all array elements have been checked.

Dry Run

Example 1.

Input: nums = [12, 345, 2, 6, 7896]

Number	Digit count	Even?	count afterward
12	2	Yes	1
345	3	No	1
2	1	No	1
6	1	No	1
7896	4	Yes	2

Final output: 2

Step 6. Analyze complexity and optimization

Let:

- (n) = number of elements in the array.
- (d) = maximum number of digits in any element.

Time complexity: (O(n x d))

We visit all n elements, and for each element, the digit-counting loop runs once per digit. For fixed-size integers, the number of digits is bounded, so this is effectively (O(n)).

Auxiliary space complexity: (O(1))

We only use a few variables: count, temp, digits, and num. We don't create another array.

NOTE:-

Can this be optimized further?

Yes, there is another neat approach using strings

```
class Solution {
    public int findNumbers(int[] nums) {
        int count = 0;

        for (int num : nums) {
            if (String.valueOf(num).length() % 2 == 0) {
                count++;
            }
        }

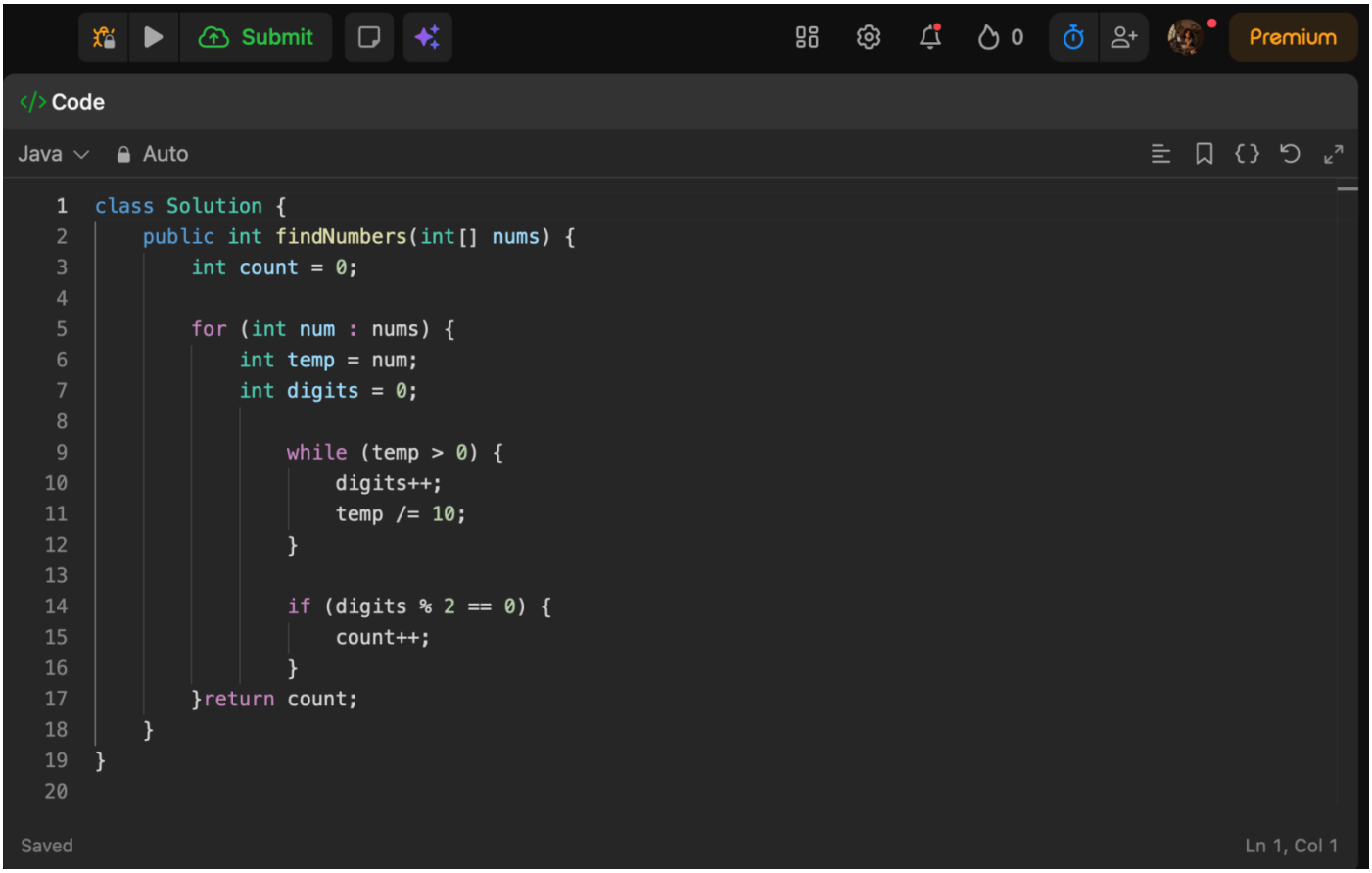
        return count;
    }
}
```

Here, String.valueOf(num) converts the number into text, and .length() gives its number of digits.

- Arithmetic approach: avoids string conversion and helps practise digit manipulation.
- String approach: shorter and very readable, but creates a string for each number.

Both are (O(n x d)) in the general digit-counting analysis and use (O(1)) auxiliary space when considering one temporary string at a time.

SCREENSHOT:-



OR

